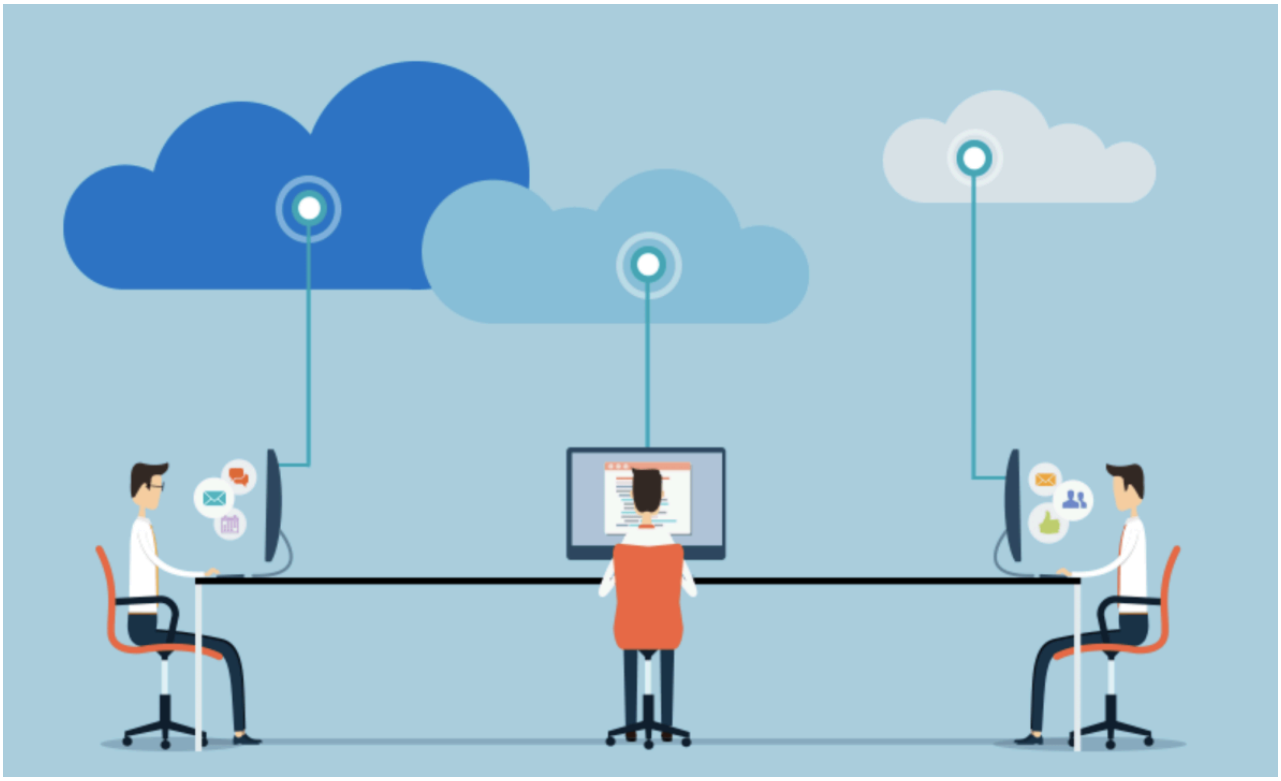
PRÁCTICA 2 SISTEMAS DISTRIBUIDOS



Javier Olozaga López
Jose Luis Palacios Medina

ÍNDICE

1. Objetivo
2. API y criterios de retorno
3. Diseño e implementación
    3.1 Servicio local (claves.c)
    3.2 Arquitectura distribuida TCP (proxy-sock.c y servidor-sock.c)
    3.3 Concurrencia y log del servidor
4. Protocolo de aplicación (independiente del lenguaje)
5. Compilación y ejecución
6. Batería de pruebas
7. Trabajo extra: benchmark de rendimiento
8. Conclusiones

1. Objetivo

En este ejercicio implementamos el servicio de tuplas del Ejercicio 1 (<key, value1, value2, value3>) sustituyendo la comunicación por sockets TCP. Mantenemos la API de claves.h y la lógica de claves.c, y únicamente se implementan proxy-sock.c y servidor-sock.c, además de adaptar el Makefile.

2. API y criterios de retorno

- 0: operación correcta
- -1: error del servicio (clave inexistente/duplicada, parámetros inválidos, etc.)
- -2: error de comunicación (no conecta / error send/recv / servidor no disponible, etc.)

Caso particular: exist(key) devuelve 1 si existe y 0 si no existe, si hay problema de comunicación devuelve -2.

Validación en proxy:

- Si IP_TUPLAS o PORT_TUPLAS no están definidos o son inválidos ->-2.
- Si parámetros fuera de límites (value1>255, N fuera de rango) ->-1.

En el proxy, devuelve -2 si falla cualquier paso de comunicación: variables de entorno no válidas, fallo al crear socket, fallo en connect(), o error en send/recv al intercambiar request/response. Si la comunicación funciona pero la operación no es válida (ej. clave inexistente), se devuelve -1.

3. Diseño e implementación

3.1 Servicio local (claves.c)

Reutilizamos la estructura dinámica del Ejercicio 1. Todas las operaciones se protegen con mutex globall, garantizando atomicidad cuando hay varios clientes concurrentes.

3.2 Arquitectura distribuida (TCP)

Cliente ->Proxy (libproxyclaves.so) ->Servidor (servidor-sock) ->libclaves.so.

- Servidor: lo lanzamos como ./servidor <PUERTO> y escucha en INADDR_ANY:<PUERTO>.
- Proxy: obtiene destino mediante variables de entorno IP_TUPLAS (IP o hostname) y PORT_TUPLAS (puerto), abre conexión TCP, envía request y espera response.
- El proxy acepta IP_TUPLAS tanto en formato IP como nombre de host, para ello resuelve el destino antes de conectar (ej. mediante resolución estándar del sistema). Esto permite ejecutar con valores como localhost o un dominio válido.

Concurrencia: el servidor atiende con hilos bajo demanda (un hilo por conexión) por simplicidad y porque es suficiente para la carga del ejercicio. Un pool reduciría overhead en cargas altas, pero complica la implementación.

Log: al recibir cada request, el servidor imprime el tipo de operación (SET/GET/MODIFY/DELETE/EXIST/DESTROY).

4. Protocolo de aplicación (lenguaje-independiente)

El protocolo no envía estructuras de C por el socket. Lo define como mensajes binarios con:

- enteros en orden de red,
- cadenas como longitud + bytes (sin \0),
- arrays como repetición de elementos.

Detalle importante (TCP): como TCP es un flujo de bytes, una llamada a recv() puede devolver menos bytes de los esperados. Por ello, el envío/recepción del protocolo se realiza acumulando hasta completar cada campo (funciones tipo send_all/recv_all). Los enteros se envían en orden de red y los mensajes se estructuran como cabecera + payload para poder reconstruirlos de forma independiente del lenguaje.

Resumen de mensajes:

Mensaje	Contenido
REQUEST	op + longitudes (key_len, value1_len) + N + value3(x,y,z) + payload (key, value1, value2)

RESPUESTA	status (0/-1) + longitudes + N + value3 + payload (value1, value2)

Flujo:
Cliente -> Proxy -> TCP ->Servidor ->TCP ->Proxy ->Cliente

Pseudocódigo (8 pasos):

1. Cliente llama a la API.
2. Proxy lee IP_TUPLAS y PORT_TUPLAS.
3. Proxy crea socket y conecta.
4. Proxy envía REQUEST(op, longitudes, datos).
5. Servidor accept y crea hilo.
6. Hilo recibe request y ejecuta operación real (libclaves.so).
7. Hilo envía RESPONSE(status, datos) y cierra conexión.
8. Proxy recibe response, cierra socket y devuelve 0/-1/-2.

5. Compilación y ejecución

Compilación:

make clean

make

Ejecución :

./servidor 4500

env IP_TUPLAS=localhost PORT_TUPLAS=4500 ./app_cliente_sock

6. Batería de pruebas

Clientes y objetivo:

Fichero	Prueba
app_cliente	funcional básico (set/get/modify/delete/exist/destroy)

app_cliente2	errores -1 (duplicadas, inexistentes, límites)
app_cliente3	concurrencia con claves únicas
app_cliente4_sock	comunicación -2 (servidor apagado / env inválidas)
app_cliente5	límites máximos válidos
app_cliente6_sock	colisión concurrente misma clave

Ejecución:

./run_tests.sh

./run_extra_tests.sh

run_tests.sh valida: funcional local, funcional TCP por IP y por hostname, configuración env inválida (-2), servidor apagado (-2), puerto incorrecto (-2), reinicio del servidor, concurrencia (claves únicas), colisión sobre misma clave y prueba prolongada de estabilidad.

7. Trabajo extra (benchmark)

Con run_extra_tests.sh (300 iteraciones/cliente): 1+-97 ops/s, 2+-195 ops/s, 4+-389 ops/s, 8+-778 ops/s. Además, prueba prolongada (4 clientes, 1000 iteraciones) sin fallos.

8. Conclusiones

Hemos migrado el servicio a TCP manteniendo la API del ejercicio 1. El servidor es concurrente, hay log de peticiones y la estructura de datos queda protegida por mutex. Las pruebas confirman funcionalidad, errores -1, errores de comunicación -2, concurrencia y estabilidad.